

CS 690: Software Development Project

Milestone 2

Architecture and Development Setup

Smart Campus Parking &
Schedule Assistant

Prepared by
Samarth Thakor and Nirmal Naik

Due October 8, 2026

GitHub repository:

<https://github.com/SamarthThakor/smart-campus-parking>

Verified starter: Working Flask application, persistent local database, and 28 passing automated tests. Source code, README, and verification records accompany this document.

1. System Architecture and Technology Stack

The application helps students choose parking for their next class. It checks the class building, permit rules, open spaces, and walking time. Parking staff maintain lot information through an administrator page. The course version does not require sensors, cameras, payments, or access to a university system.

1.1 Layered client-server design

The design follows Lecture 4: separate user interaction, application services, and database responsibilities [2]. The Flask backend is one application with separate modules for accounts, schedules, parking administration, and recommendations. This keeps the two-person project manageable while allowing each module to be tested independently.

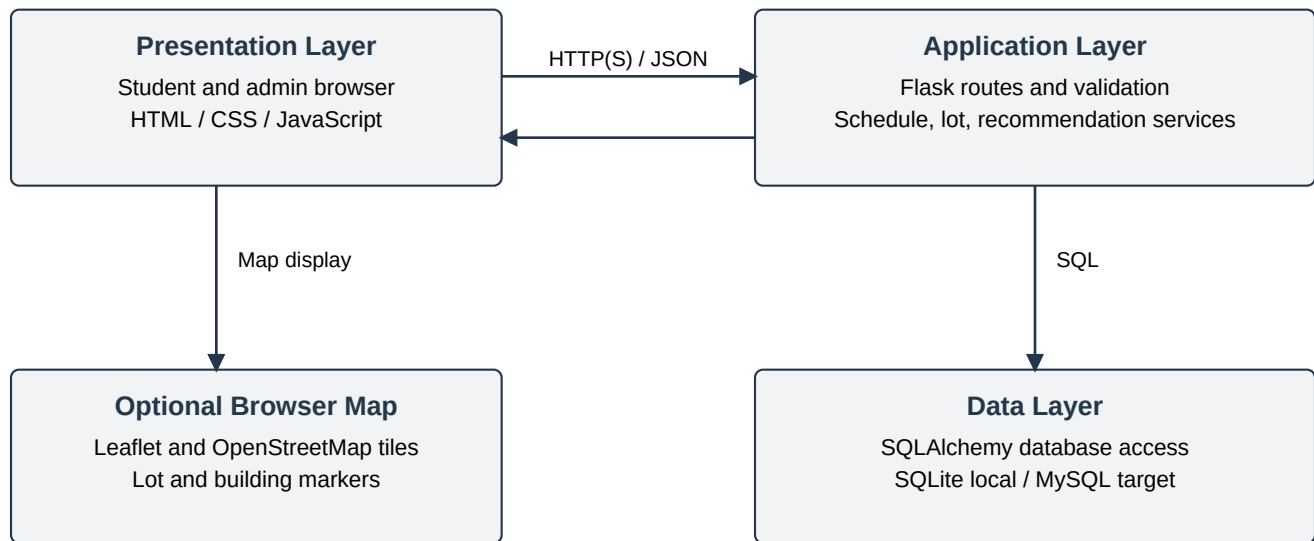


Figure 1. Component design. The verified starter uses local HTTP and SQLite. Flask serves the frontend and API from the same origin; HTTPS and MySQL are the deployment targets. The browser never connects directly to the database.

1.2 Selected technologies

Component	Technology and justification
Frontend	HTML, CSS, and vanilla JavaScript for forms, schedule pages, and the dashboard.
Backend	Python with Flask for account handling, JSON routes, validation, and recommendation logic.
Database	SQLAlchemy stores structured records in SQLite for the verified starter. MySQL remains the deployment target. The local setup creates the schema; later schema evolution will use migrations.
Optional maps	Leaflet with OpenStreetMap tiles. Walking times come from stored lot-to-building estimates; a map library alone does not calculate walking routes.
Planned hosting	AWS Elastic Beanstalk for Flask and Amazon RDS for MySQL, as proposed in Milestone 1. Deployment has not started.

1. System Architecture and Technology Stack

1.3 How the components work together

Scenario 1 - finding parking: James signs in and saves a weekly schedule and permit. The dashboard asks Flask for the next class and a parking recommendation. The service reads the database, excludes unsuitable lots, and returns the best lot with its space count, walking time, and last update time.

Scenario 2 - updating parking: Jordan signs in as an administrator and closes Lot C. The API validates the role and saves the change in a database transaction. The next student request reads the updated lot status and excludes Lot C.

These flows keep the interfaces connected to the users and scenarios from Milestone 1. This applies Lecture 3's approach of developing features around user goals [1]. John Doe, the first-year student persona, uses the same student workflow with clear building names and permit information.

1.4 Design decisions and tradeoffs

Concern	Project decision
Maintainability	Keep route handling, recommendation rules, and database models in separate modules. Use defined inputs and return values so changes remain local.
Performance	Use one backend and one shared relational database. Index student schedule lookups and lot-permit pairs. Avoid extra service-to-service requests for this course scope.
Reliability	Use constraints and transactions for lot updates. Show a clear empty result when no eligible lot exists. Database failure produces an error, not a misleading parking recommendation.
Security	Hash passwords, use protected session cookies, validate input on the server, and enforce ownership and administrator roles. Use parameterized database access and keep credentials out of Git.
Usability	Provide one sign-in flow, readable class and lot names, and specific form errors. Show when availability was updated. Counts indicate reported availability and do not reserve spaces.
Availability	Begin with one deployed application and a managed database. This is simpler to maintain but does not provide a custom failover design. Additional redundancy can be considered later.

1.5 Deployment boundary

The cloud plan places Flask on Elastic Beanstalk and MySQL on RDS. The database should accept connections only from the application environment, and public browser traffic should use HTTPS. The verified quickstart uses SQLite to avoid a separate database installation. SQLAlchemy defines a shared logical schema, and a MySQL schema export is included for review. MySQL integration and cloud deployment remain unverified.

Lecture 4 discusses security as a concern across layers and the cost of adding distributed components [2]. For this project, a modular application with a shared database offers a practical balance. Splitting every feature into a separate service would add deployment and data-consistency work before the core workflows are complete.

1. System Architecture and Technology Stack

1.6 Requirement-to-design traceability

Lecture 3 distinguishes broad scenarios from individual user stories [1]. The table below connects the Milestone 1 backlog to the modules and checks in the starter. Story identifiers follow the submitted Milestone 1 document (CS690_Milestone1.pdf), which contains 15 user stories.

Milestone 1 requirement	Component and interface	Starter evidence
US-01-02: account and permit	Account routes; user and permit_type tables.	Login and registration API, hashed passwords, permit API, access tests.
US-03-04: schedule and next class	Class routes and get_next_class(); class_session and building tables.	Schedule CRUD API, week-boundary test, next-class dashboard.
US-05-08: suitable parking	recommend_parking(); parking_lot, lot_permit, walk_estimate.	Tests exclude full, closed and wrong-permit lots; verify ranking and empty results.
US-09-12: administrator updates and access	Admin routes and update_lot(); transaction-backed updates.	Browser controls and API; closure, invalid-count, role and permit-rule tests.
US-14-15: arrival time and alternatives	Recommendation response; sorted eligible candidates.	API returns parked-by time and up to two alternatives. Main recommendation and arrival time appear in the dashboard.
US-13: map display	Optional Leaflet browser component.	Deferred. The starter does not require a map to select a lot.

1.7 Starter scope and quality goals

Verified now: database initialization, seeded student/admin accounts, login, recommendation API, administrator updates, schedule operations, and automated checks. The source package separates models, services, routes, templates, and static assets.

Next interface work: display the accepted permit-type list for US-06 (the starter currently returns an eligibility flag), and add browser forms for registration, schedule editing, and permit selection. The APIs are implemented; the recorded tests focus on backend behavior. The current student page displays seeded schedules, and automated browser testing is not included. MySQL integration, versioned migrations, optional maps, and AWS deployment are later steps.

Acceptance criteria: a closed lot must disappear from the next recommendation; invalid updates must leave stored data unchanged; a student must never modify another user's class or administer parking; no-match conditions must return a readable reason.

Performance target: a future campus-scale dataset should return a recommendation within two seconds under normal load. This is a target, not a measured load-test result. This milestone verifies correctness and basic module communication.

2. Module Interfaces and Data Structure Design

2.1 Database relationships

A class entry belongs to one student and one building. A lot may accept multiple permit types. Each lot-to-building pair has one stored walking-time estimate.

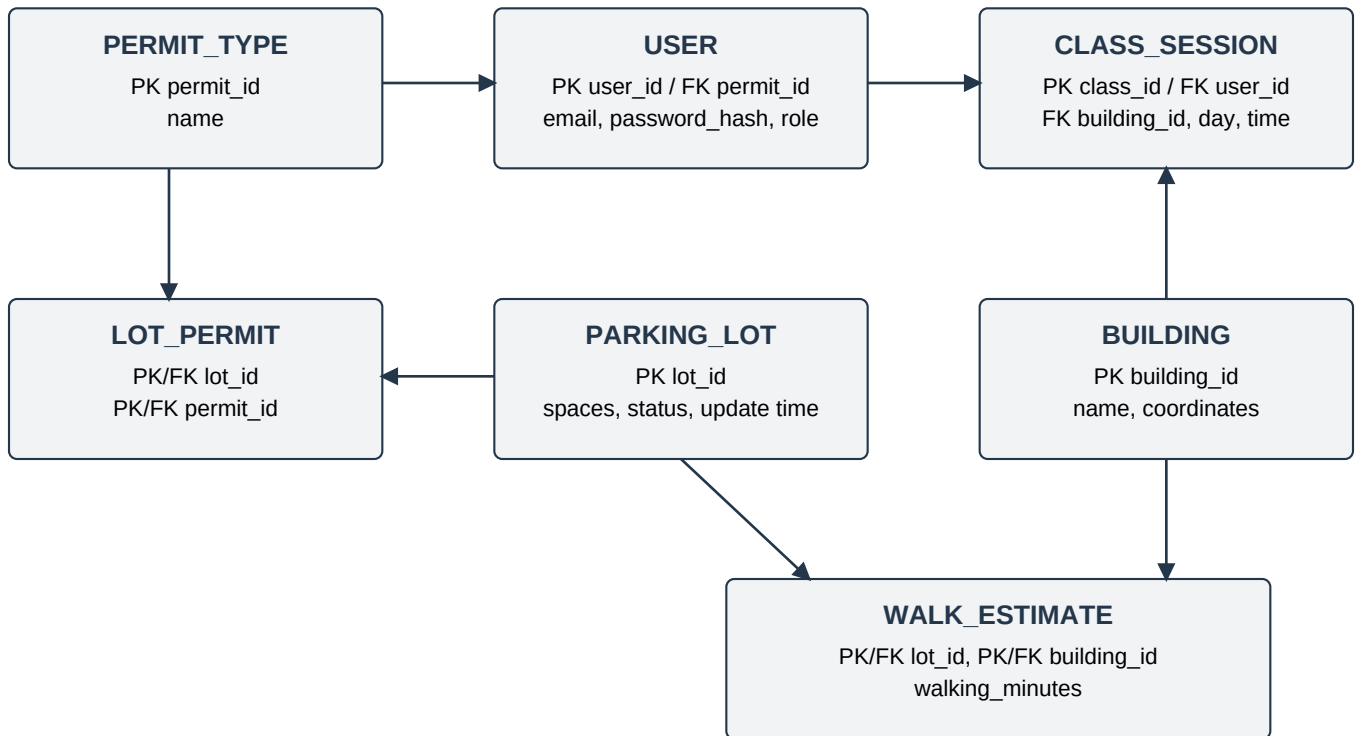


Figure 2. Each arrow runs from a parent table to a child table in a one-to-many relationship. PK = primary key; FK = foreign key. USER.permit_id is optional until a student selects a permit.

2.2 Main fields

Table	Fields and types
permit_type	permit_id INT PK; name VARCHAR(50), unique.
user	user_id INT PK; email VARCHAR(255), unique; password_hash VARCHAR(255); role VARCHAR(10) with student/admin check; permit_id INT nullable FK.
class_session	class_id INT PK; user_id and building_id INT FKs; title VARCHAR(100); weekday INT (0-6); start_time and end_time TIME.
building	building_id INT PK; name VARCHAR(100); latitude and longitude DECIMAL(9,6).
parking_lot	lot_id INT PK; name VARCHAR(100); capacity and available_spaces INT; status VARCHAR(10) with open/closed check; coordinates DECIMAL(9,6); updated_at VARCHAR(32), ISO 8601 UTC.
lot_permit	lot_id and permit_id INT FKs; composite PK (lot_id, permit_id).
walk_estimate	lot_id and building_id INT FKs; walking_minutes INT; composite PK (lot_id, building_id).

2. Module Interfaces and Data Structure Design

2.3 Data rules and recommendation logic

Required fields cannot be null, except the student's initial permit selection. Space counts must be between zero and capacity. Walking minutes must be positive. Class end time must be later than start time. Referenced buildings and permits cannot be deleted while in use; deleting a student removes that student's class entries.

Weekly class times use the configured campus time zone. Weekdays run from 0 (Monday) to 6 (Sunday); request times use HH:MM. The next-class search checks the next seven days. The recommendation service keeps open lots with spaces, an accepted permit, and a walking estimate to the class building. It sorts by walking minutes ascending, available spaces descending, and lot ID ascending to resolve ties consistently.

If no class, permit, or eligible lot is available, the response explains the reason. Optional features include two alternate lots and a parked-by time calculated as class start minus walking time minus a five-minute buffer. The first release uses maintained estimates rather than a live route service.

2.4 Student and account routes

These JSON contracts are implemented in the starter. S1 means the parking scenario; S2 means the administrator-update scenario. All student routes use the signed-in user's identity rather than accepting another user's ID.

Route and required input	Return values and scenario connection
POST /api/auth/register email, password	201: student ID. 400: invalid fields; 409: duplicate email. Starts the account workflow for S1. Registration cannot create an admin.
POST /api/auth/login email, password	200: user ID, role, and session cookie. 401: invalid credentials. Opens James's saved information (S1).
POST /api/auth/logout Session and CSRF token	204: session cleared. Ends account access for S1 and S2.
PATCH /api/me/permit permit_id	200: saved permit. 400: unknown permit. Limits recommendations to permitted lots (S1).
GET /api/buildings and GET /api/permits No body	200: lists of IDs and names for the class form and permit selector (S1).
GET /api/classes No body	200: the student's class list (S1).
POST /api/classes title, weekday, start_time, end_time, building_id	201: created class with ID. 400: invalid fields or time range. Adds a destination and time for S1.
PATCH /api/classes/{id} One or more editable class fields	200: updated class. 400: invalid final values; 404: missing or not owned. Keeps S1 schedule information accurate.
DELETE /api/classes/{id} No body	204: deleted. 404: missing or not owned. Removes outdated schedule entries (S1).

2. Module Interfaces and Data Structure Design

2.5 Recommendation and administrator routes

Route and required input	Return values and scenario connection
GET /api/classes/next No body; server uses current time	200: next class and start timestamp with time-zone offset, or class: null. Identifies James's next destination (S1).
GET /api/recommendations No body	200: recommended lot, available spaces, permit eligibility, walking minutes, update time, and optional alternatives. Empty results include a reason (S1).
GET /api/admin/lots No body	200: lot details, accepted permit IDs, and update times. Lets Jordan inspect information (S2).
PATCH /api/admin/lots/{id} available_spaces and/or status	200: saved lot and updated_at. 400: invalid count or status; 404: missing lot. Changes availability and closures (S2).
PUT /api/admin/lots/{id}/permits permit_ids: list of valid IDs	200: replacement permit list. 400: unknown or duplicate IDs; 404: missing lot. Supports the Milestone 1 permit-rule story. An empty list permits no students.
GET /api/health No body	200: status: ok when the API and database are reachable; 503: unavailable otherwise. Supports setup verification.

2.6 Internal service contracts

Function and arguments	Expected return and purpose
get_next_class(conn, user_id: int, now: datetime)	Dictionary or None. Uses a time-zone-aware timestamp and returns the next weekly class occurrence (S1).
recommend_parking(conn, user_id: int, now: datetime)	Dictionary with ranked lots or an empty result and reason: no_upcoming_class, permit_required, or no_eligible_lot (S1).
update_lot(conn, lot_id: int, changes: dict)	Updated lot record inside the route transaction. The route checks the admin role and commits; invalid input raises ValueError, missing lot raises LookupError (S2).

2.7 Shared errors and authorization

Protected routes return 401 without a valid session. Administrator routes return 403 for student accounts. Changing requests require CSRF protection. Invalid JSON or fields return 400 with an error code and readable message. Unexpected server errors return 500 without exposing database details. The API checks class ownership before reads or changes.

Session and token setup

GET /api/auth/csrf takes no input and returns 200 with csrf_token. The browser keeps its session cookie and sends X-CSRF-Token on writes, including login. Login returns a rotated token. GET /api/me returns 200 with the current user ID, email, role, and permit, or 401 without a session. Both support the student and administrator workflows.

3. Build Automation and Continuous Integration Setup

3.1 Version control and review workflow

The source package includes .gitignore and a GitHub Actions workflow. The public GitHub repository contains the source and workflow. The team uses main for reviewed code and feature/name or fix/name branches for individual work. Commits use feat:, fix:, test:, or docs: prefixes. Each pull request states the change and test results; the other member reviews before merge.

Samarth leads frontend and coordination. Nirmal leads backend, database, QA, and cloud setup. Both members review and test complete workflows. Passing checks and one approval should be required through repository settings after publication. The hosted workflow passed on October 6, 2026. Branch protection is a planned enforcement setting; the branching and review rules above define the team policy.

3.2 Implemented local automation

scripts/check.py compiles the Python modules and runs pytest. It stops with a nonzero status if either stage fails. With --watch, it monitors application, test, and dependency files and reruns checks automatically when source content changes. Tests create isolated temporary SQLite databases and do not change the demonstration database.

```
python scripts/check.py
python scripts/check.py --watch
```

Stage	Implemented check	Recorded result
Syntax	Compile app and test modules.	Passed. JavaScript syntax also checked separately.
Behavior	Run pytest against fresh temporary databases.	28 passed; no failures.
Source update	Start watch mode, change a source file, observe automatic rerun.	Change detected; all 28 tests passed again.
Live communication	Send actual HTTP requests to the running Flask server.	200 health, 200 admin update, recommendation changed from Lot B to Lot A.

3.3 Verified GitHub Actions workflow

.github/workflows/ci.yml runs on pushes and pull requests. It checks out the repository, selects Python 3.12, installs the pinned development requirements, and runs the same check.py command. Repository permissions are read-only for the job. This starter workflow uses SQLite; a MySQL service job can be added after MySQL integration is verified.

3.4 Test coverage and limits

The 28 cases cover class CRUD and ownership, weekly scheduling, permit eligibility, closed/full lots, ranking ties, no-match results, invalid counts, administrator authorization, CSRF, registration, password hashing, logout, database constraints, and safe seeding. The suite follows the Flask testing approach of isolated fixtures and request clients [4].

The local source-change automation and GitHub-hosted workflow have both completed successfully. These results demonstrate the starter's behavior; they do not establish production load capacity, a completed browser workflow for every feature, or a successful MySQL/AWS deployment.

4. Development Environment and Starter Code Execution

4.1 Reproducible local setup

The included README contains the full quickstart, configuration options, demo credentials, API conventions, and troubleshooting notes. Python 3.12.14 was used for local verification on Windows. Dependencies are pinned in requirements.txt and requirements-dev.txt.

1. Get the project. Clone the public repository linked on the cover and open the folder containing README.md. The README includes the clone command and complete setup instructions.

2. Create an environment and install dependencies.

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements-dev.txt
Copy-Item .env.example .env
```

3. Initialize and seed the local database. SQLite is included with Python. These commands create the tables and add fictional users, class sessions, buildings, permits, lots, and walking estimates:

```
.venv\Scripts\python -m flask --app app:create_app init-db
.venv\Scripts\python -m flask --app app:create_app seed-demo
```

The database is stored in instance/parking.db. Initialization preserves existing tables; it does not migrate schema changes. Seeding refuses to overwrite existing user data. The MySQL option and generated schema are documented separately and have not been integration-tested.

4. Start the application. Open <http://127.0.0.1:5000> after starting the server:

```
.venv\Scripts\python -m flask --app app:create_app run
```

5. Run checks in another terminal.

```
.venv\Scripts\python scripts/check.py
.venv\Scripts\python scripts/check.py --watch
```

4.2 Demonstration accounts and sequence

Role	Local demonstration credentials
Student	alex@example.test / DemoParking123!
Administrator	jordan@example.test / DemoParking123!

Sign in using the student demo account (alex@example.test): Lot B should show 18 reported spaces and a five-minute walk. Sign out and sign in as Jordan, close Lot B, and save. Sign back in using the student demo account: Lot A should now be recommended with an eight-minute walk. Reopen Lot B afterward to reset the demonstration. All accounts and locations are fictional, and these demo credentials are not suitable for deployment.

4.3 Configuration

DATABASE_URL selects the database; CAMPUS_TIMEZONE controls weekly scheduling. SECRET_KEY should be set to a random value for shared use. APP_HTTPS=1 enables Secure cookies after HTTPS is configured. Local .env files, virtual environments, and database files are excluded from Git. Production serving, secrets, backups, and rate limiting remain deployment work.

4. Development Environment and Starter Code Execution

4.4 Recorded verification evidence

Verification was performed locally on October 6, 2026. The source package includes docs/local-checks.txt and docs/http-demo.json. The latter records real HTTP requests to a running server at port 5055; the normal README quickstart uses port 5000. Lot B was restored to open after the recorded demonstration.

Action	Observed response	What it demonstrates
GET /api/health	HTTP 200; status = ok	Flask and the local database are reachable.
Student recommendation	HTTP 200; Lot B; 18 spaces; 5-minute walk	The API combines class, permit, lot and walking-time records.
Admin closes Lot B	HTTP 200; lot_id = 2; status = closed	Authorized state change persisted successfully.
Student requests again	HTTP 200; Lot A; 10 spaces; 8-minute walk	The next request uses the updated database state.
Source-change check	28 tests passed before and after a detected source change	Automatic local verification runs on code updates.
GitHub Actions	Build and test run #1: success; 28 tests passed	The starter installs and verifies on a clean hosted Ubuntu runner.

4.5 Repository and hosted verification

The repository contains executable starter code, pinned dependencies, tests, README instructions, local automation, the GitHub workflow, a MySQL schema export, and captured verification records. Virtual environments, local secrets, and runtime database files are excluded.

GitHub verification: Build and test, run #1 completed successfully on October 6, 2026. The Ubuntu runner installed the pinned dependencies, compiled the modules, and passed all 28 tests in 5.96 seconds. Tested source commit: 58f830c. The recorded result is included in docs/github-verification.json.

Repository access: the source repository is public, so the instructor and teammate can view and clone it without an invitation. The owner manages teammate write access separately. The cover contains the live repository link.

References

[1] *Lecture 3: Features, Scenarios, and Stories*. CS 690 course lecture slides, provided as Lecture_3.pdf. Personas and scenarios: PDF pp. 5-6 and 13-19; user stories: pp. 22-24.

[2] *Lecture 4: Software Architecture*. CS 690 course lecture slides, provided as Lecture_4.pdf. Quality tradeoffs: PDF pp. 5-14; layered architecture and security: pp. 19-23; distribution and technology choices: pp. 26-32.

[3] *CS 690 Milestone 2: Architecture and Development Setup*. Assignment handout, due October 8, 2026.

[4] Pallets. *Testing Flask Applications*. Flask testing documentation. Accessed October 6, 2026.

[5] SQLAlchemy. *Defining Constraints and Indexes*. SQLAlchemy documentation. Accessed October 6, 2026.

[6] GitHub. *Quickstart for GitHub Actions*. GitHub Actions documentation. Accessed October 6, 2026.